The 8 Great Ideas in Architecture

1. **Design for Moore's Law:** Anticipate where technology will be when the chip finishes design.
2. **Use abstraction to simplify design:** Hide lower-level details (ISA hides hardware from software).
3. **Make the common case fast:** Optimize what happens most often (immediate operands).
4. **Performance via parallelism:** Do multiple things at once (multicore processors).
5. **Performance via pipelining:** Overlap instruction execution (assembly line).
6. **Performance via prediction:** Guessing is often faster than waiting for a sure answer.
7. **Hierarchy of memories:** Small/fast cache near CPU; large/slow disk further away.
8. **Dependability via redundancy:** Provide backups for failing components.

Defining Performance

Response Time (Latency): How long to do *one* task.

Throughput (Bandwidth): Total work done per unit time.

Measuring Time:

- **Elapsed (Wall-clock):** Total response time, includes I/O, OS overhead, other apps. (System perf).
- **CPU Time:** Time spent processing a specific job. Discounts I/O time. (CPU perf).

Amdahl's Law

Concept: The overall speedup is limited by the fraction of the time the enhancement can actually be used.

$$T_{new} = \frac{T_{affected}}{\text{Improvement Factor}} + T_{unaffected}$$

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{S}}$$

How to use: (f) is the % of time the enhancement is used. (S) is the speedup of that specific feature. If a feature is used 50% of the time ($f = 0.5$), the max overall speedup is 2x, even if $S = \infty$.

CPU Clocking & The Iron Law

Clock Period (T_c): Duration of a clock cycle (e.g., 250ps).

Clock Frequency (f): Cycles per second (e.g., 4.0 GHz).

Conversion: $f = 1/T_c$ (1ns $= 10^{-9}$s, 1GHz $= 10^9$Hz)

The Iron Law of Performance

$$\text{Clock Cycles} = \text{IC} \times \text{CPI}$$

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times T_c = \frac{\text{IC} \times \text{CPI}}{f}$$

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instructions}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

- **IC (Instruction Count):** Determined by program, ISA, and Compiler.
- **CPI (Cycles Per Instruction):** Determined by CPU hardware/architecture.
- T_c **(Clock Cycle Time):** Determined by physics/manufacturing.

You cannot optimize one metric without checking the others. Faster clock (f) might increase CPI. Less instructions (IC) might require a slower clock.

Advanced CPI & MIPS

Weighted Average CPI: Different instruction classes (ALU, Load, Branch) take different amounts of cycles.

$$\text{Global CPI} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \frac{\text{IC}_i}{\text{Total IC}} \right)$$

How to use: Multiply the CPI of a specific instruction type by its percentage of occurrence in the code, then sum them up.

Pitfall: MIPS

$$\text{MIPS} = \frac{\text{IC}}{\text{Time} \times 10^6} = \frac{f}{\text{CPI} \times 10^6}$$

Why it's bad: MIPS ignores the *complexity* of instructions. It varies between programs on the same CPU and cannot reliably compare different ISAs.

Relative Performance

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

"X is n times faster than Y"

$$n = \frac{\text{Perf}_X}{\text{Perf}_Y} = \frac{\textbf{Time}_Y}{\textbf{Time}_X}$$

How to use: Always put the SLOWER machine's time (the larger number) on top to get a speedup > 1.

Registers

Size: 32 registers, 64-bits wide each.

- **Doubleword:** 64-bit data.
- **Word:** 32-bit data.

ISA & Design Principles

Principle 1: Simplicity favors regularity

- All arithmetic operations have exactly 3 operands (1 destination, 2 sources).
- *Why:* Regularity makes hardware implementation simpler, enabling higher performance at lower cost.

Principle 2: Smaller is faster

- LEGv8 has a strict 32 × 64-bit Register File instead of keeping variables in massive RAM.
- *Why:* Electrical signals travel faster over smaller physical distances (32 registers vs millions of memory locations).

Principle 3: Make the common case fast

- Immediate operands (`ADDI X1, X2, #4`).
- *Why:* Small constants are common in programming (loops, array indexing). Encoding them directly in the instruction avoids a slow memory load.

Principle 4: Good design demands good compromises

- Keeping all instructions exactly 32-bits long, but allowing different formats (R, D, I, B, CB) inside those 32 bits.

Basic Blocks & Compilation

Basic Block: A sequence of instructions with:

1. No embedded branches (except at the very end).
2. No branch targets (except at the very beginning).

Compilers easily optimize basic blocks because they guarantee straight-line execution.

Inverted Logic for If-Statements: When compiling `if (i == j) f=g+h;`, Assembly checks the *opposite* condition to jump over the block: `SUB X9, i, j`
`CBNZ X9, Skip` (If $i - j \neq 0$, jump past the ADD)
`ADD f, g, h`
`Skip: ...`

Memory

Hierarchy: Registers → Cache → Main Mem → Disk.

Byte Addressed: Every memory address identifies an 8-bit byte. Because a Doubleword is 64 bits (8 bytes), **array indices must be multiplied by 8** to find the byte offset!

Alignment: LEGv8 does not strictly require word alignment for data, but does for instructions and the stack.

Data Transfer Instructions:

- **LDUR (Load):** Memory → Register.
- **STUR (Store):** Register → Memory.
- *Syntax:* `LDUR Rt, [Rn, #offset]`
- *Calculation:* Address $= Rn + offset$.

Arithmetic & Immediates

Base Arithmetic: `ADD X1, X2, X3`
Subtraction: `SUB X1, X2, X3`
Immediate Math: `ADDI X1, X2, #10`
Negative Immediates: LEGv8 machine code has **no** `SUBI` opcode. The Assembler translates an assembly `SUBI X1, X2, #5` into `ADDI X1, X2, #-5` (using 2's complement).

Binary Numbers & Math

Unsigned Integers: Range is 0 to $+2^n-1$.
Signed Integers (2's Complement):

- Range: -2^{n-1} to $+2^{n-1}-1$.
- **Sign Bit (MSB):** 0 = Positive, 1 = Negative.

Sign Extension: When moving an 8-bit or 32-bit number into a 64-bit register, you must preserve the value.

- **Positive:** Pad left with 0s. (`LDURB` zero-extends).
- **Negative:** Pad left with 1s. (`LDURSB` sign-extends).

Logical Operations

Used for bitwise manipulation rather than standard math.

- **LSL (Logical Shift Left):** Shifts bits left, fills with 0s. Mathematically **multiplies by** 2^i.
- **LSR (Logical Shift Right):** Shifts bits right, fills with 0s. Mathematically **divides by** 2^i (unsigned).
- **AND:** Output 1 if *both* are 1. Used for **Masking** (forcing specific bits to 0 while keeping others).
- **ORR:** Output 1 if *either* is 1. Used to **Set** bits to 1 without affecting others.
- **EOR (XOR):** Output 1 if inputs are *different*. Used to **Invert** (NOT) bits by XORing with a string of 1s.

Base Conditional Branches

- `CBZ register, L1`: If (register == 0), goto L1.
- `CBNZ register, L1`: If (register != 0), goto L1.
- `B L1`: Unconditionally goto L1.

Condition Codes (Flags)

Instructions ending in **S** (`ADDS`, `SUBS`, `ANDIS`) set 4 condition flags based on the ALU result:

- **N (Negative):** Result had a 1 in the MSB.
- **Z (Zero):** Result was exactly 0.
- **V (Overflow):** Signed overflow occurred.
- **C (Carry):** Unsigned carryout occurred.

Using Flags for "If" Statements: Instead of just checking for zero, use `SUBS` to set flags, then branch based on the result.

- **Pattern:** `SUBS XZR, X1, X2` (Performs X1-X2, throws away answer into XZR, but sets NZVC flags).

Branch Flag Instructions (Green Card):

- `B.EQ`: Equal (Z=1)
- `B.NE`: Not Equal (Z=0)
- `B.LT`: Less Than (*Signed*)
- `B.LE`: Less Than or Equal (*Signed*)
- `B.GT`: Greater Than (*Signed*)
- `B.GE`: Greater Than or Equal (*Signed*)
- `B.LO`: Less Than (*Unsigned*)
- `B.HI`: Greater Than (*Unsigned*)

Signed vs Unsigned Trap: `-1 < +1` in Signed. But in Unsigned, `-1` is `0xFFFFFF...` which is $> +1$.

Branch Offset Calculation

All branch addresses use **PC-Relative Addressing**.
Assembly → Machine Code:

- The offset placed in the binary instruction is the number of **INSTRUCTIONS** (lines of code) to skip.
- **Forward jump:** Positive number.
- **Backward jump:** Negative number (Use 2's comp!).
- *Example:* If `CBZ` is line 1, and the target is line 4, the offset is +3.

Machine Code → Memory Address (CPU logic):

- Because every instruction is 4 bytes (32 bits), the CPU must calculate the *Byte Offset*.
- Byte Offset = Instruction Offset × 4.
- **Hardware shift:** The CPU shifts the binary instruction offset **Left by 2** to multiply by 4.
- Target PC = Current PC + (Offset $\ll 2$).

Instruction Formats

Every LEGv8 instruction is exactly **32 bits**.

R-Format (Register Math: ADD, SUB, AND)
shamt: Shift amount (usually `000000` unless LSL/LSR).

I-Format (Immediate Math: ADDI, SUBI)
I-Format opcodes are 10 bits. The remaining 1 bit becomes part of the immediate. Zero-extended.

Instruction Syntax & Behavior Guide

In Assembly text, the Destination is written FIRST (Left). In Machine Code binary, the Destination is slotted LAST (Right).

1. Arithmetic & Logical (R-Format)

- `ADD Rd, Rn, Rm` → `Rd = Rn + Rm`
- `SUB Rd, Rn, Rm` → `Rd = Rn - Rm`
- `AND Rd, Rn, Rm` → `Rd = Rn & Rm`
- `ORR Rd, Rn, Rm` → `Rd = Rn | Rm`
- `EOR Rd, Rn, Rm` → `Rd = Rn ^ Rm`

2. Immediate Math (I-Format)

- `ADDI Rd, Rn, #imm` → `Rd = Rn + imm`
- `SUBI Rd, Rn, #imm` → `Rd = Rn - imm`
- *Note:* `#imm` is a 12-bit unsigned number.

3. Data Transfer (D-Format)

- `LDUR Rt, [Rn, #offset]` → `Rt = Memory[Rn + offset]`
- `STUR Rt, [Rn, #offset]` → `Memory[Rn + offset] = Rt`
- `Rt` is Destination for LDUR, but **Source** for STUR.

4. Shifts (R-Format)

- `LSL Rd, Rn, #shamt` → Shift `Rn` left by `shamt` (Fills w/ 0s. Multiplies by 2^{shamt}).
- `LSR Rd, Rn, #shamt` → Shift `Rn` right by `shamt` (Fills w/ 0s. Divides by 2^{shamt}).

5. Branching (CB / B Formats)

- `CBZ Rt, Label` → If `Rt == 0`, jump to `Label`.
- `CBNZ Rt, Label` → If `Rt != 0`, jump to `Label`.
- `B Label` → Unconditionally jump to `Label`.
- `B.cond Label` → Jump to `Label` if condition flag is met.
- *Address:* PC-Relative. **Instruction** offset.

The "Times 8" Rule (Arrays & Memory)

Memory is Byte-Addressed. Registers hold 64-bit Doublewords (8 Bytes). To find the address of an array element, **multiply the index by 8**.
Scenario A: Constant Index (e.g., A[3])

- Offset = $3 \times 8 = 24$.
- Assembly: `LDUR X9, [X20, #24]`

Scenario B: Variable Index (e.g., A[i] where i is in X22)

- Multiply i by 8 using shift: `LSL X10, X22, #3` (X10 is now Byte Offset)
- Add to Base Address: `ADD X10, X10, X25` (X10 is now Physical Addr)
- Load data: `LDUR X9, [X10, #0]`